

A QUICK ONE BEFORE WE START



Every filter is either on or off. That is one search.

15 filters.
How many combinations
can a user make?

32,768

Every filter you add doubles the number.

200?

1,000?

5,000?

And that's with each filter just on or off.
Ours take up to fifty values each.

BEFORE WE START

Filter over 400 million rows, any mix of filters, in under a second.

What would you build, and what would it cost you?

Option A

Postgres, with the
right indexes

Option B

Elasticsearch,
straight away

Shout a number

What would it cost
per month?

ELASTIC AI CONNECT · 26 SEPTEMBER 2026

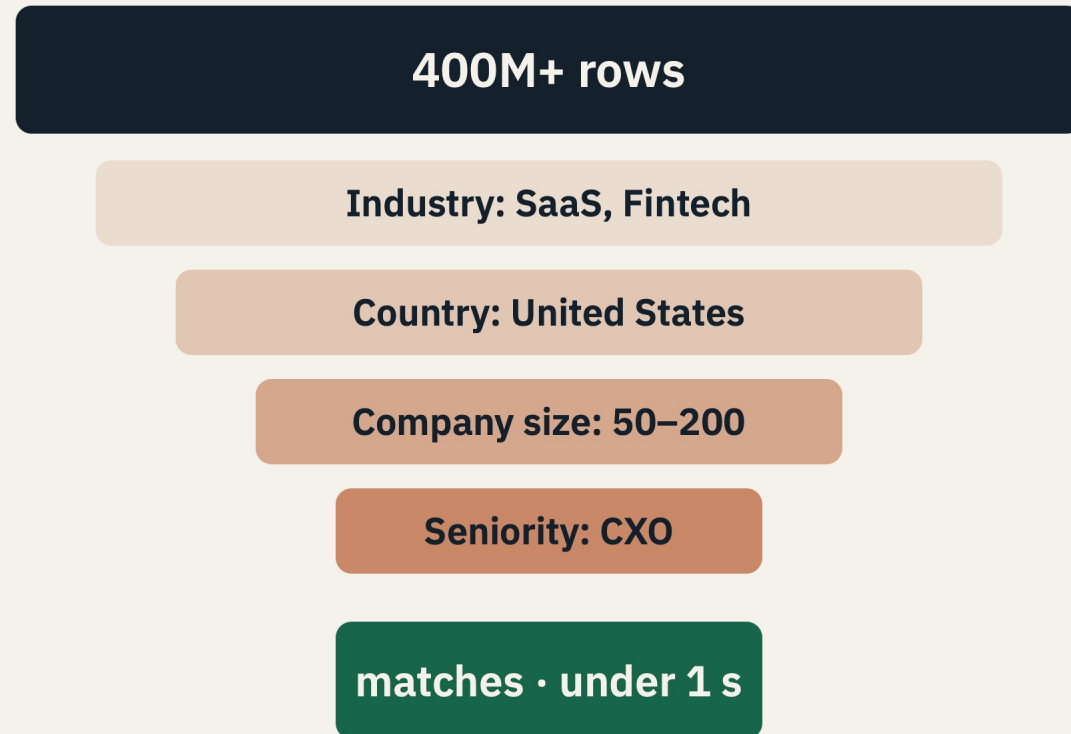
Taming the search bill

How we optimized Elastic as a bootstrapped startup

Rudra Narayan Gupta · Co-Founder & CTO, GrowthToolKit.io

THE PROBLEM

We needed a filter engine



Users stack filters

They apply 5 to 8 at a time, each with many values, over 400M+ rows.

The combinations explode

That's the 32,768 we started with. Any filter can be missing, so no one index fits.

It has to feel instant

Under 1 s worst case, ideally 500–800 ms.

We sized our first cluster by guesswork

CREATE CLUSTER

Instance type

?

Number of nodes

?

Storage per node

?

GB

Estimated cost

?

/ month

Create cluster

THREE YEARS LATER

The instance type?

Honestly, I don't remember.

The bill?

That, I'll never forget.

ATTEMPT 1 • MANAGED OPENSEARCH



1

2

3

4

5

6

7

Seven days on that cluster. How much?

Shout a number.

ATTEMPT 1 • MANAGED OPENSEARCH

~\$0

1

2

3

4

5

6

7

of AWS credits, gone in seven days.

So we shut it down.

Plan B: make Postgres do it

1 The data was already there

2 No new server, no new bill

3 So we tried GIN

ROUGHLY WHAT WE TRIED

```
ALTER TABLE prospects
  ADD COLUMN doc tsvector
  GENERATED ALWAYS AS (
    to_tsvector('simple',
      coalesce(title, '') || ' ' ||
      coalesce(company, ''))
  ) STORED;

CREATE INDEX prospects_doc_gin
  ON prospects USING GIN (doc);
```

ATTEMPT 2 · POSTGRES

Watch Postgres run one search

country: US

industry: SaaS, Fintech

company size: 50-200

seniority: CX0

matches found

ROWS IN THE TABLE

- Ruled out by the index
- Candidate: check the row
- Checked, not a match
- Match

One square stands for millions of rows.

QUERY TIME

40+ s

60+ s without GIN

Why it didn't work for us

POSSIBLE FILTER COMBINATIONS

Any of the 15 can be on or off, and each takes up to ~50 values.

Words and values in one query

Bio and skills need word matching. Seniority, company size and year need exact values and ranges. One search asks for both at once.

You have to guess

We indexed what we thought mattered. Users don't always pick those filters.

Big table, slow answers

At 400M+ rows: over a minute, still 40+ s with GIN. We needed under 1 s.

Postgres is still great. This just wasn't its job.

The same search in Elasticsearch

```
country: US
industry: SaaS, Fintech
company size: 50-200
seniority: CX0
```

in all four

THE FOUR BARS

■ Documents this filter matches

ROWS IN THE TABLE

■ In all four: a match

□ Not a match

One square stands for millions
of rows.

QUERY TIME

<300 ms

usually 200–300 ms
rarely 1–5 s on very
heavy searches

Why Elasticsearch fits this problem

ONE SEARCH = ONE BOOL FILTER

```
GET prospects/_search
{
  "query": { "bool": { "filter": [
    { "terms": { "industry": ["saas", "fintech"] } },
    { "term": { "country": "US" } },
    { "range": { "size": { "gte": 50, "lte": 200 } } }
  ] } }
}
```

Every field has its own lists

Words, exact values, numbers, dates: each gets the kind of list that suits it. Nothing is left un-indexed.

Combining is comparing lists

Four filters means comparing four lists and keeping what is in all of them. The work depends on the shortest list, not the size of the table.

No ranking, just yes or no

We are not asking which row is the best match. Every row is in or out, so all the scoring maths is skipped.

Popular filters are remembered

Filters people use constantly are kept ready in memory, so the next search reuses them.

ATTEMPT 3 · SELF-HOSTED ELASTICSEARCH

So I learned to run Elasticsearch myself

OUR FIRST SETUP

Where	DigitalOcean, US
Version	Elasticsearch 8.16
Servers	2 × ~32 GB RAM, 8–12 vCPU
Sync	Background job on every write
Backups	Daily snapshots to S3
Cost	\$300–500 / month

TWO NODES, EACH WITH A FULL COPY



Each box is a slice of the data. P is the live slice, R is its copy on the other node.

ATTEMPT 3 · ONE NODE

Then we noticed one good server was enough

WHAT WE SAW ON TWO NODES

- Both nodes sitting mostly idle
- Searches at 200–300 ms
- Each node held a full copy of the data
- B2B traffic: steady, no big spikes

WHAT WE CHANGED

Moved to one bigger server and set replicas to 0.
Searches stayed at 200–300 ms.

ONE NODE, ONE COPY OF EACH SHARD

node-1

P0

P1

P2

6 cores · 64 GB RAM · 1 TB disk

Cost: a fraction of that

`number_of_replicas: 0`

no second copy inside Elasticsearch

If that node dies: daily S3 snapshots, and Postgres is still the source of truth.

The last 7 days on that one server

CPU

<1%



7-day average, peaking
around 0.75%

MEMORY

35 GB



of 64 GB in use, flat all week

DISK

~470 GB



used, under half the disk

Plenty of room before a second node earns its keep.

The settings that matter on one node

```
# elasticsearch.yml
discovery.type: single-node

# every index
PUT prospects/_settings
{ "index": { "number_of_replicas": 0 } }

# backups: Kibana → Snapshot and Restore
# repository: AWS S3 · schedule: daily
```

Replicas to 0

A copy never lives on the same machine as the original. Leave replicas at 1 on a single node and the copy has nowhere to go, so the cluster sits yellow forever.

replicas: 1 → yellow

replicas: 0 → green

Daily snapshots to S3

Kibana runs them on a schedule. With no second copy, this is the backup.

Leave the rest to the OS

The filesystem cache is what keeps searches fast. Fast local disks help too.

What one node costs you

What you give up	How we cover it
Automatic failover	Restarts and upgrades in a low-traffic window
A second copy	Daily S3 snapshots; Postgres has the data
Scaling out	Scale up if needed; plenty of room today

When I wouldn't run one node

If search going down for an hour breaks a promise to customers.

If the data outgrows one machine, or the traffic stops being predictable.

RECAP

The whole journey, in four stops

01

Managed OpenSearch

Right tool, but ~\$3,000
of AWS credits in a week.

TOO EXPENSIVE

02

Back to Postgres

No new bill, but searches
took 40+ seconds.

TOO SLOW

03

Self-hosted, two nodes

Right tool, \$300–500
a month.

WORKED, BUT OVERSIZED

04

Self-hosted, one node

Same speed, a fraction
of the cost.

TODAY

THE NUMBERS

Before and after

SETUP	COST	TYPICAL SEARCH
Managed OpenSearch	~\$3,000 / week	not measured
Postgres + GIN	no new bill	40+ s
Self-hosted, two nodes	\$300–500 / mo	200–300 ms
Self-hosted, one node (today)	a fraction of that	200–300 ms

100x + faster than Postgres,
at a fraction of the
two-node cost

BACK TO THE QUESTION

Filter over 400 million rows, any mix of filters, in under a second.

What would you build, and what would it cost you?

What we built

Elasticsearch

After Postgres couldn't keep up with our filters

Where it runs

One node

Self-hosted, 6 cores, 64 GB RAM

What it costs

A fraction

of the two-node cost

Elasticsearch was never the expensive part. The way we ran it was.

QUESTIONS

Thanks!

Happy to take questions, and to hear your own search-bill stories.

400 million rows · any mix of filters · 200–300 ms · one node

Rudra Narayan Gupta · Co-Founder & CTO, GrowthToolKit.io

rudra@growmeorganic.com · [linkedin.com/in/rudra-gupta-6381951a0](https://www.linkedin.com/in/rudra-gupta-6381951a0) · x.com/rudragupta9506 · rudragupta.xyz